

Brilliance EMS

HR Employee Module — API Documentation

Base URL: <http://localhost:5000> (or Cloudflare tunnel URL)

Prefix: </api/hr/employees> · Related: </api/applications>

Generated: 2026-08-02

All routes below require:

Authorization: Bearer <jwt_token>

Allowed roles: hr | admin

Standard response:

```
{ "success": true, "message": "...", "data": { ... } }
```

Error response:

```
{ "success": false, "message": "...", "errors": ["..."] }
```

Employee :id can be MongoDB _id OR empId (e.g. EMP001).

Soft-delete sets status=Deleted and disables login (User.isActive=false, isDeleted=true).

0. Auth — get JWT

POST /api/auth/login

Auth: No auth

Headers: Content-Type: application/json

Body (JSON):

```
{  "email": "hr@brilliance.com",  "password": "your-password",  "role": "hr"}
```

Success response (data):

```
{  "token": "<jwt>",  "user": { "id", "name", "email", "role", ... } }
```

Note: Use data.token as Bearer token for all HR Employee APIs.

1. Employee object shape (normalized)

Every create / get / update / list item returns this shape inside data.employee or data.employees[]:

```
{  "id": "<Employee MongoId>",  "name": "Ali Khan",  "empId": "EMP001",  "role": "Employee",  "designation": "Software Engineer",  "dept": "Engineering",  "department": "Engineering",  "joined": "2026-08-01T00:00:00.000Z",  "dateOfJoining": "2026-08-01T00:00:00.000Z",  "email": "ali@brilliance.com",  "phone": "03001234567",  "status": "Active",           // Active | Inactive | Deleted  "branch": "Lahore",  "manager": "Sara Ahmed",  "managers": [    { "name": "Sara Ahmed", "relationship": "Primary" }  ],  "salary": 85000,  "address": { "street": "...", "city": "...", "country": "... },  "emergencyContact": {    "name": "...", "phone": "...", "relation": "..."  },  "bank": {    "bankName": "...", "accountNumber": "...", "iban": "..."  },  "assets": [    {
```

```

      "id": "...", "name": "Laptop", "tag": "LT-01",
      "status": "Assigned", "assignedOn": "..."
    }
  ],
  "documents": [
    {
      "id": "...", "name": "CNIC.pdf", "type": "cnic",
      "uploadedAt": "...", "url": "/uploads/documents/..."
    }
  ],
  "initial": "AK",
  "userId": "<User MongoId>"
}

```

2. Employees CRUD — /api/hr/employees

2.1 List employees

GET /api/hr/employees

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Query: department?=string status?=Active|Inactive|Deleted search?=string page?=1 limit?=20

Success response (data):

```

{
  "employees": [ /* Employee objects */ ],
  "pagination": {
    "page": 1, "limit": 20, "total": 42, "pages": 3
  }
}

```

Note: Excludes Deleted by default unless status=Deleted. Also backfills Employee profiles for legacy staff Users.

2.2 Create employee

POST /api/hr/employees

Auth: Bearer JWT required

Roles: hr | admin

Headers: Content-Type: application/json | Authorization: Bearer <token>

Body (JSON):

```

{
  // % % REQUIRED % %
  "name": "Ali Khan",
  "email": "ali@brilliance.com",
  "phone": "03001234567",
  "password": "TempPass123", // min 8 characters
  "designation": "Software Engineer",
  "dateOfJoining": "2026-08-01",
  "salary": 85000, // number > 0

  // % % OPTIONAL % %
  "empId": "EMP010", // omit !' auto EMP001 / EMP002 ...
  "role": "employee", // default: employee
  "department": "Engineering", // default: "General"
  "branch": "Lahore", // default: "Head Office"
  "manager": "Sara Ahmed", // User id | empId | email | name
  "address": { "street": "", "city": "", "country": "" },
  "emergencyContact": { "name": "", "phone": "", "relation": "" },
  "bank": { "bankName": "", "accountNumber": "", "iban": "" },
  "documentName": "Offer Letter",
  "applicationId": "<ApplicationId>" // marks application Selected
}

```

Success response (data):

```

{ "employee": { /* full Employee incl. generated empId */ } }

```

Note: Simplified Add Employee form. Only 7 fields required. empId never required from client. Unique email + empId. Defaults: role=employee, department=General, branch=Head Office.

2.3 Get employee by id / empId

GET /api/hr/employees/:id

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Mongo _id OR empId (EMP001)

Success response (data):

```

{ "employee": { /* full Employee object */ } }

```

2.4 Partial update

PATCH /api/hr/employees/:id

Auth: Bearer JWT required

Roles: hr | admin

Headers: Content-Type: application/json | Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Body (JSON):

```
{
  "name": "Ali Khan Updated",
  "phone": "03001112233",
  "designation": "Senior Engineer",
  "department": "Engineering",
  "branch": "Karachi",
  "role": "employee",
  "dateOfJoining": "2026-07-15",
  "manager": "<manager id or name>",
  "salary": 95000,
  "status": "Active",
  "address": { "street": "...", "city": "...", "country": "..."},
  "emergencyContact": { "name": "...", "phone": "...", "relation": "..."},
  "bank": { "bankName": "...", "accountNumber": "...", "iban": "..."}
}
```

Success response (data):

```
{ "employee": { /* updated Employee object */ } }
```

Note: All fields optional. password is NOT accepted here (returns 400). Syncs User + ProfileCompletion bank/emergency where applicable.

2.5 Activate

POST /api/hr/employees/:id/activate

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Success response (data):

```
{ "employee": { "status": "Active", ... } }
```

Note: Sets Employee.status=Active, User.isActive=true. Cannot activate Deleted employees.

2.6 Deactivate

POST /api/hr/employees/:id/deactivate

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Success response (data):

```
{ "employee": { "status": "Inactive", ... } }
```

Note: Sets Employee.status=Inactive, User.isActive=false (login blocked).

2.7 Soft delete

DELETE /api/hr/employees/:id

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Success response (data):

```
{ "employee": { "status": "Deleted", ... } }
```

Note: Soft-delete: status=Deleted, User.isDeleted=true, isActive=false. Removes manager assignments. Login disabled.

3. Documents — /api/hr/employees/:id/documents

GET /api/hr/employees/:id/documents

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Success response (data):

```
{
  "documents": [
    {
      "id": "...",
      "name": "CNIC.pdf",
      "type": "cnic",
      "uploadedAt": "2026-08-03T00:00:00.000Z",
      "url": "/uploads/documents/...."
    }
  ]
}
```

POST /api/hr/employees/:id/documents

Auth: Bearer JWT required

Roles: hr | admin

Headers: Content-Type: multipart/form-data | Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Body (multipart/form-data):

file (required) – pdf | doc | docx | jpeg | png | webp (max 10MB)
type (optional) – e.g. cnic | contract | other
name (optional) – display name (defaults to original filename)
documentName (optional) – alias for name

Success response (data):

```
{
  "document": { "id", "name", "type", "uploadedAt", "url" },
  "documents": [ /* full list */ ]
}
```

Note: File stored under /uploads/documents/. Also mirrored on User.documents for employee portal.

4. Assets — /api/hr/employees/:id/assets

GET /api/hr/employees/:id/assets

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Success response (data):

```
{
  "assets": [
    {
      "id": "...",
      "name": "MacBook Pro",
      "tag": "LT-42",
      "status": "Assigned",
      "assignedOn": "2026-08-01T00:00:00.000Z"
    }
  ]
}
```

POST /api/hr/employees/:id/assets

Auth: Bearer JWT required

Roles: hr | admin

Headers: Content-Type: application/json | Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Body (JSON):

```
{
  "name": "MacBook Pro",           // required
  "tag": "LT-42",
  "status": "Assigned",           // optional (default Assigned)
  "assignedOn": "2026-08-01"     // optional (default now)
}
```

Success response (data):

```
{
  "asset": { "id", "name", "tag", "status", "assignedOn" },
  "assets": [ /* full list */ ]
}
```

5. Managers link

GET /api/hr/employees/:id/managers

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Mongo _id OR empld

Success response (data):

```
{
  "managers": [
    { "name": "Sara Ahmed", "relationship": "Primary" },
    { "name": "Bilal Raza", "relationship": "Secondary" }
  ]
}
```

Note: Uses ManagerEmployeeAssignment. Falls back to User.manager / Employee.manager name if no assignment rows.

6. Hire from interviewed candidate

POST /api/applications/:id/select

Auth: Bearer JWT required

Roles: hr | admin

Headers: Authorization: Bearer <token>

Params: :id = Application Mongoid

Success response (data):

```
{
  "application": { /* status: Selected, ... */ },
  "employee": { /* normalized Employee object */ },
  "user": { /* safe User object */ }
}
```

Note: Application must be status Interview. Creates/updates Employee + sets User.role=employee. Use returned employee to open Add Employee / profile screens.

7. Common HTTP status codes

200 — Success (GET/PATCH/POST activate|deactivate|DELETE)

201 — Created (POST employee | document | asset)

400 — Validation / business rule (e.g. password on PATCH, wrong application status)

401 — Missing / invalid JWT

403 — Role not hr/admin, or account suspended

404 — Employee / application not found

409 — Email or empId already in use

8. Example cURL

Login

```
curl -X POST http://localhost:5000/api/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"hr@brilliance.com","password":"secret12","role":"hr"}'
```

List employees

```
curl http://localhost:5000/api/hr/employees?search=Ali&page=1 \
-H "Authorization: Bearer <token>"
```

Create employee (required fields only)

```
curl -X POST http://localhost:5000/api/hr/employees \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"name":"Ali Khan","email":"ali@brilliance.com","phone":"03001234567","password":"TempPass123","designation":"Software Engineer","dateOfJoining":"2026-08-01","salary":85000}'
```

Upload document

```
curl -X POST http://localhost:5000/api/hr/employees/EMP001/documents \
-H "Authorization: Bearer <token>" \
-F "file=@./cnic.pdf" \
-F "type=cnic" \
-F "name=CNIC Front"
```

Add asset

```
curl -X POST http://localhost:5000/api/hr/employees/EMP001/assets \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{"name":"Laptop","tag":"LT-01","status":"Assigned"}'
```

Select interviewed candidate

```
curl -X POST http://localhost:5000/api/applications/<appId>/select \
-H "Authorization: Bearer <token>"
```

Brilliance EMS — HR Employee Module API · React Native contracts: hrPortalService.js, hrManagerService.js, applicationService.js